Hospital AMR — Nurse App, Robot Interface & Fleet Management Dashboard

Objective

Design and develop the full operator-facing software layer for a hospital autonomous mobile robot (AMR) delivery system. The system consists of three interfaces: a nurse-facing mobile app for requesting and tracking deliveries, an on-robot touchscreen interface for staff to authenticate and retrieve payloads, and a web-based Fleet Management System (FMS) dashboard for operations staff to monitor and control the entire robot fleet. All three interfaces must communicate over the hospital Wi-Fi network and be accessible from anywhere within the hospital without requiring a fixed workstation.

System Overview

The hospital AMR is a compact autonomous robot designed for high-frequency intra-hospital logistics transporting lab samples, medications, and consumables between wards, pharmacies, and laboratories. The robot carries up to 20 kg across multiple electronically locked compartments. Each compartment is assigned to a specific delivery mission and can only be unlocked by the authorised recipient using a passcode. The system shall support a fleet of multiple robots operating simultaneously within the hospital.

The Fleet Management System (FMS) is the central on-premise backend responsible for mission allocation, fleet monitoring, charging coordination, traffic management, map storage, and incident logging. The nurse app and robot interface communicate with the FMS over the hospital network. The developer is responsible for developing the operator-facing software stack, including the nurse application, robot touchscreen interface, web dashboard, backend services, database, and communication layer.

Project Requirements

The nurse-facing mobile app is used by staff to create and track delivery requests. A nurse logs in, selects the pickup location, destination ward, and payload category (lab sample, medication, or consumable), and submits the request. The FMS assigns an available robot and the app shows live mission progress through the following

notification states: Mission Accepted, Robot Assigned, Robot En Route, Robot Arrived, Compartment Unlocked, Mission Completed, Mission Delayed, and Mission Failed. When the robot arrives, the authorised recipient receives a one-time compartment passcode. Entering the passcode through either the nurse application or the robot touchscreen unlocks only the assigned compartment while all other compartments remain secured. The app must function entirely over the hospital internal network and Wi-Fi with no dependency on the public internet.

The on-robot touchscreen interface is the local UI mounted on the robot body, used by ward staff at the point of delivery. It displays the robot's current destination, mission status, and any action required from nearby staff in plain language (e.g. "Awaiting pickup at Ward 3B" or "Enter passcode to open compartment"). Staff enter a passcode to unlock the specific compartment assigned to their mission; all other compartments remain locked. The robot confirms successful handoff before closing the delivery task. The interface must communicate the robot's motion intent before it moves — displaying a visual indicator and playing an audio alert — so nearby staff and patients are not startled. The interface must be legible in bright corridor lighting.

Every delivery mission shall progress through the following lifecycle states:

Created → Assigned → En Route to Pickup → Loading → En Route to Destination → Awaiting Recipient → Completed / Failed / Cancelled

Mission state transitions must be visible in both the nurse application and FMS dashboard in real time.

The FMS web dashboard (React, desktop-first, accessible from any device on the hospital network) is used by operations staff and administrators to oversee the fleet. Fleet monitoring must include real-time robot locations on a floor plan map with path visualisation, robot status, Battery Percentage, Charging Status, safety state (Normal / Protective Stop / E-Stop), and connection health. Operators must be able to issue per-robot commands: pause mission, resume mission, cancel mission, return to charger, reassign missions, and prioritise urgent deliveries. The dashboard must also provide a mission queue view showing active, pending, completed, and failed missions with prioritisation and reassignment controls.

The incident management section must display fault reports, emergency stops, safety events, mission failures, compartment access events, and operator actions — all with timestamps and filtering capabilities. A dedicated system health page must show each

robot's online status, last heartbeat timestamp, Wi-Fi signal strength, Battery Health, Charging Status. An analytics dashboard must provide operational metrics including deliveries per day, average mission duration, battery usage statistics, mission success rate, and fleet performance metrics. The dashboard must support CSV export of mission reports, audit reports, incident reports, and utilisation reports for a selected date range.

The backend must support real-time telemetry streaming over WebSocket. Telemetry messages from the robot to the FMS include: robot online/offline, current position (X, Y, heading), battery percentage, charging state, mission acceptance, mission start, mission completion, safety state, fault reports, network status, and heartbeat. Commands from the FMS to the robot include: create mission, pause mission, resume mission, cancel mission, return to charger, dynamic reroute, and traffic management instructions. The developer must implement this full message protocol between the FMS backend and robot agent end-to-end.

The system must implement role-based access control with the following roles. Nurses can create requests, track missions, and unlock compartments. Pharmacists can manage medication deliveries and unlock medication compartments. Lab technicians can manage sample deliveries and unlock sample compartments. Operations staff can monitor the fleet, manage missions, and control robots. Administrators have full access including user management, location configuration, map management, analytics, and report export.

The FastAPI backend is responsible for authentication, authorisation, mission management, audit logging, telemetry processing, WebSocket communication, report generation, and fleet coordination APIs. The database must persist users, missions, fleet telemetry, audit logs, maps, user activity logs, incident reports, and analytics data. Mission records must be searchable by date range, ward, robot ID, mission status, and staff member. Every compartment access event must be recorded with user identity, timestamp, robot ID, mission ID, and compartment ID. The audit trail must be immutable and queryable.

The system must be accessible from anywhere within the hospital. The FMS dashboard and nurse app must be reachable from any device connected to the hospital Wi-Fi — nurses' personal phones, ward tablets, and operations workstations without installing native apps. The backend must be deployed on the on-premise server with a local domain (e.g. http://amr.hospital.local) so no public internet is required. WebSocket

connections must support automatic reconnection with a visible status indicator when the network drops. The system shall gracefully handle robot disconnections, backend restarts, and temporary network outages.

Design the complete system in Figma before implementation. Designs must use high-contrast status colours for robot states, be legible in bright hospital lighting, and follow accessibility guidelines for clinical environments.

Deliverables

1. Figma design file covering all screens for all three interfaces: nurse app, robot touchscreen, and FMS dashboard.
2. Nurse mobile app with login, delivery request, real-time mission tracking, compartment passcode view and mission history.
3. Robot touchscreen interface showing mission state, passcode entry, delivery confirmation, and motion intent display.
4. FMS web dashboard (React, desktop-first, accessible from any hospital Wi-Fi device) with real-time fleet map, robot status cards, command panel, mission queue management, incident log, system health page, analytics dashboard, and CSV report export.
5. FastAPI backend with JWT authentication, role-based access, mission lifecycle management, full AMR-FMS message protocol over WebSocket, and immutable audit log.
6. Documentation covering system architecture, message protocol specification, local network deployment guide, and API reference.

References

The following links are provided for general understanding of hospital delivery robots and should not be considered implementation requirements.

1. https://relayrobotics.com/relay-delivery-robots-for-hospitals
2. https://workdrive.zohoexternal.in/external/writer/cc8d211a62ba29013c2901f6650a5eca6839493fb2ea97e23cf84d7b0eb5c1f2